PS-033

DNS-Based Discovery of Diagnostic Services

Category: Cloud Computing & IoT

Team . TwoPointThree

AIORI Hackathon 3

October 10, 2026

Problem Description

Diagnostic services are found through brittle, unauthenticated configuration

- Latency probes and traceroute responders are located by hardcoded IPs or proprietary registries that break when endpoints move.
- Plain DNS answers can be forged or altered, so a client cannot tell valid metadata from bogus data.
- Finding an endpoint must not be mistaken for being allowed to invoke it, or for trusting it.
- Malformed records and unavailable services must fail safely and visibly.

1 Unsigned lab.test zone

3 mock diagnostic services

1 discovery client

Background

Three existing building blocks turn DNS into a discovery channel

1 DNS-AID

Publishes service metadata in DNS so clients can discover endpoints dynamically [1].

example

`_index._agents.lab.test`

2 SVCB records

One query returns target, port, ALPN and address hints for a service [2].

example

`port=8443 alpn=h2`

3 DNSSEC and DANE

Signatures give origin authentication and integrity; TLSA can pin the TLS key [3][5].

example

`RRSIG, DNSKEY, DS, TLSA`

DNS-AID gives a verifiable transport for metadata, not a trust signal [1]

Literature & RFC Review

Only the standards this prototype actually relies on

Source	What it defines	Our use
DNS-AID draft-02 [1]	Discovery use cases, SVCB keys, trust rules	Pinned subset: use cases 1 and 2
RFC 9460, SVCB [2]	Service binding: target, port, ALPN	Publish and parse service metadata
RFC 4033, RFC 9364, DNSSEC [3][4]	Chain of trust, signed RRsets	Validate every discovered record
RFC 6698, RFC 7671, DANE [5][6]	TLSA binds a service to its TLS key	Authenticate the TLS endpoint
RFC 6763, RFC 8552, DNS-SD [7][8]	Service labels, underscored names	_index._agents leaf naming
RFC 8615, well-known URIs [9]	Standard path for descriptors	Fetch descriptor, verify cap-sha256

Pinned scope: draft-02 (27 May 2026), Known Agent and Known Organization use cases. Out of scope: domain control validation, EDNS hints, registry and search.

Proposed Solution

Discover, validate, invoke, with integrity and trust kept separate

1. **Publish:** Sign the zone; add SVCB (and TLSA) per service.
2. **Query:** Fetch the _index._agents record, then each service.
3. **Validate:** DNSSEC chain, then the metadata digest.
4. **Invoke:** Authenticated call; the service decides access.
5. **Report:** Classify: valid, bogus, malformed, unavailable.

Layer 1: metadata integrity

DNSSEC and cap-sha256 show the record came from the zone owner, unaltered. They cannot show the service is safe.

Layer 2: authorisation and trust

Enforced by the service (mTLS or token) on each request, plus policy outside DNS. DNS never grants it.

Implementation Status & Future Work

Today·s milestone: Docker-based DNS lab with an unsigned authoritative zone

Implemented in the current milestone

BIND runs in Docker and serves the lab.test zone on 127.0.0.1:5300.

Three service names are configured: latency.lab.test, traceroute.lab.test and lossjitter.lab.test.

Each service name resolves to 127.0.0.1; the SOA response is used to check authoritative service (AA flag).

DNS configuration and zone files are maintained in the team GitHub repository.

Future work

Sign the lab.test zone with DNSSEC and configure Unbound as a validating resolver.

Choose and publish service metadata using SVCB parameters (including port=) or TXT records.

Add TLSA/DANE endpoint binding, parallel SVCB and TLSA queries, and TTL-based caching.

Extend tests to cover bogus signatures, malformed metadata and unavailable services.

Solution Architecture & Design

Component view: a DNS plane for integrity, a service plane for authorisation

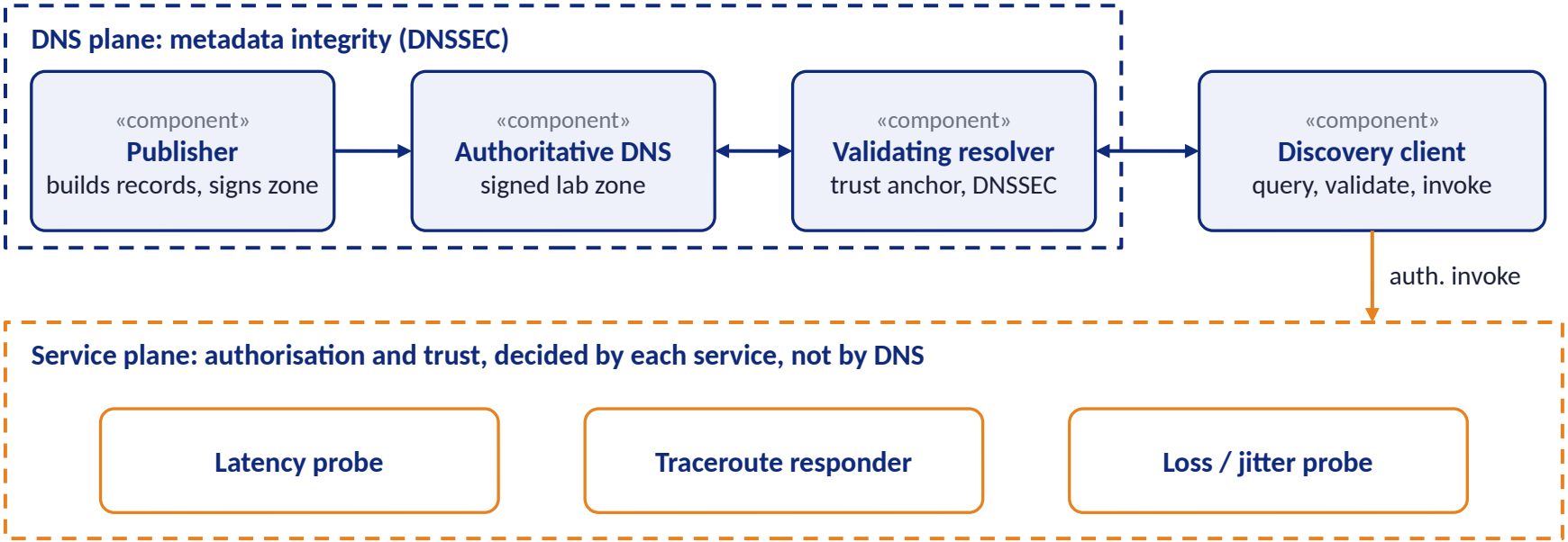
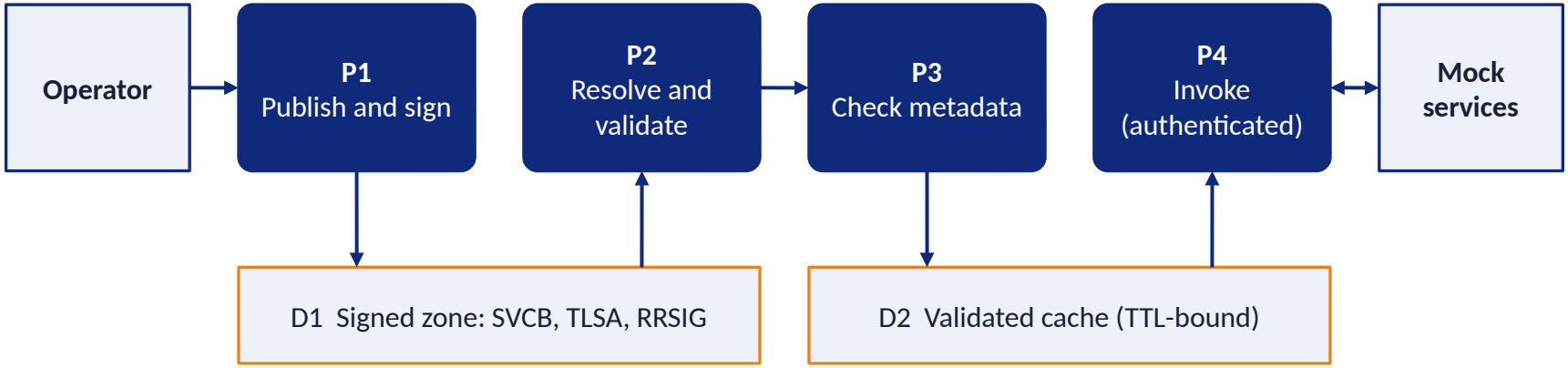


Figure: System architecture (UML component view); faults are injected at the zone and the services.

Data Flow Diagram

Level-1 flow: what data moves, where it is stored, where it is rejected



P2 rejects BOGUS DNS data

P3 rejects MALFORMED metadata

P4 reports UNAVAILABLE service

Figure: Level-1 data flow diagram (DFD)

Deployment Timeline

Incremental implementation: establish discovery first, then add validation and trust

Completed / current milestone

Run the Docker BIND container and serve the unsigned lab.test zone.

Publish latency, traceroute and lossjitter names and verify their A records.

Check the authoritative SOA response and commit the DNS configuration to GitHub.

Next milestone: DNSSEC and resolver validation

Sign the zone, retain the required public trust material, and configure Unbound.

Test successful validation and rejection of deliberately bogus DNS data.

Later work

Finalize the service-record format and JSON index fields.

Integrate the discovery client and mock services; add TLSA, parallel lookups and TTL caching.

Complete negative tests, capture evidence, and prepare the final report and demonstration.

Bibliography

[1] J. Mozley, N. Williams, B. Sarikaya, R. Schott, J. Damick. “DNS for AI Discovery.” Internet-Draft draft-mozleywilliams-dnsop-dnsaid-02, 27 May 2026 (work in progress).

[2] B. Schwartz, M. Bishop, E. Nygren. “Service Binding and Parameter Specification via the DNS (SVCB and HTTPS Resource Records).” RFC 9460, Nov. 2023.

[3] R. Arends et al. “DNS Security Introduction and Requirements.” RFC 4033, Mar. 2005.

[4] P. Hoffman. “DNS Security Extensions (DNSSEC).” BCP 237, RFC 9364, Feb. 2023.

[5] P. Hoffman, J. Schlyter. “The DNS-Based Authentication of Named Entities (DANE) Transport Layer Security (TLS) Protocol: TLSA.” RFC 6698, Aug. 2012.

[6] V. Dukhovni, W. Hardaker. “The DNS-Based Authentication of Named Entities (DANE) Protocol: Updates and Operational Guidance.” RFC 7671, Oct. 2015.

[7] S. Cheshire, M. Krochmal. “DNS-Based Service Discovery.” RFC 6763, Feb. 2013.

[8] D. Crocker. “Scoped Interpretation of DNS Resource Records through ‘Underscored’ Naming of Attribute Leaves.” BCP 222, RFC 8552, Mar. 2019.

[9] M. Nottingham. “Well-Known Uniform Resource Identifiers (URIs).” RFC 8615, May 2019.